National Emission Standards for Hazardous Air Pollutants (NESHAP) Maximum Achievable Control Technology (MACT) Floor Analysis for Example Industry Units for Final Rule

OAR-OCAP-NRD-EDAB and/or author names here

November 17, 2025

Introduction

Section 112 of the Clean Air Act (CAA) requires the the U.S. Environmental Protection Agency (EPA) establish National Emission Standards for Hazardous Air Pollutants (NESHAP) for the control of the hazardous air pollutants (HAP) emitted from both new and existing sources in a source category. These standards must reflect the maximum degree of reduction in emissions that is achievable. The minimum level of control is referred to as the "Maximum Achievable Control Technology (MACT) floor." The method for determining the MACT floor as the minimum control level allowed for establishing standards for a NESHAP is defined for both new and existing sources by CAA Section 112(d)(3). For new sources, the MACT floor cannot be less stringent than the emission control that is achieved in practice by the best-controlled similar source. For existing sources, the MACT floor cannot be less stringent than the average emission limitation achieved by the best-performing 12 % of existing sources for source categories with 30 or more sources, or the best-performing 5 sources for source categories with fewer than 30 sources.

The purpose of this memorandum is to present the methodology and the results of the MACT floor analysis for the NESHAP source category Example Industry. This MACT floor analysis uses data collected in a nationwide survey of Example Industry unit owners and operators conducted by the EPA in *2025* under the Information Collection Request for NESHAP for Example Industry.

MACT Floor Analysis Hazardous Air Pollutants

The specific chemicals, compounds, or groups of compounds designated to be HAP are listed in *CAA section 112(b)*. From this HAP list, the following HAP, among others, were identified to be emitted from Example Industry units and were included in the MACT floor analysis: *chemical X, chemical Y, chemical Z. . . .*

MACT Floor Analysis subcategories

Under *CAA section 112(d)(1)*, the EPA has the discretion to ". . . distinguish among classes, types, and sizes of sources within a category or subcategory in establishing. . . " standards. When separate subcategories are established, a MACT floor is determined separately for each subcategory. To determine whether the EGU source category warrants subcategorization for the MACT floor analysis, the EPA reviewed Example Industry design, operating information, and air emissions data compiled in the ICR data set and other information collected by the Agency for development of the NESHAP for this source category. Based on this review, the EPA concluded that there are significant design and operational differences in Example Industry that affect the levels of emissions of certain HAP to justify subcategorizing Example Industry for the purpose of establishing MACT floors.

For the MACT floor analysis, the following Example Industry subcategories were defined:

- **Subcategory 1.** *describe it here and how/why it was determined*

- **Subcategory 2.** *describe it here and how/why it was determined*

Calculating Emission Values used in the MACT Floor Analysis

The EPA received emissions data from industry respondents under the Example Industry MACT ICR in a number of different units of measure and converted these units of measure to a common *insert measurement units.*

If any outlier analysis or subsequent QA/QC was done on the emissions data prior to MACT floor analysis, please include it here.

MACT Floor Analysis Methodology

The first step in the MACT floor analysis for each subcategory and HAP (or surrogate) was to rank all of the Example Industry units in the subcategory (for which there were emissions data in the ICR data set) for the pollutant type by emission level (lowest to highest). From this ranking, a MACT floor pool of sources was identified for determining the minimum control level allowed for the MACT floor, consistent with the criteria defined for new and existing sources by CAA Section 112(d)(3). For the new-source MACT floors, the best-controlled similar source was identified for which there were individual source test run data in the ICR data set. The selection of top performing sources for the existing source MACT floor was performed using the `MACT_existing()` function in the `UPLforOAR` `R` package version 0.1.0, with details in Appendix C. The selection of the best performing source for the new source MACT floor was performed using the `MACT_new()` function in the `UPLforOAR` `R` package version 0.1.0, with details in appendix C.

For the existing-source MACT floors, selection of the MACT floor pool size (i.e., number of units to be included in the determination of the average emission limitation value) was determined on an individual subcategory basis as described below.

- **Subcategory 1.** *do any subcategories need different handling?*
- **Subcategory 2.** *do any subcategories need different handling?*

Probability distribution Selection

The next step in the MACT floor analysis was to incorporate data variability into the calculations of the applicable MACT floor limits for the subcategories using a 99% upper prediction limit (UPL) approach. Specifically, the MACT floor limit is an UPL calculated with the Student's t-distribution using the `qt()` function in the `R` `stats` software. The Student's t-distribution has also been used in other EPA rulemakings (e.g., NESHAP for Portland Cement, NSPS for Hospital/Medical/Infectious Waste Incinerators [HMIWI], NESHAP for Industrial, Commercial, and Institutional Boilers and Process Heaters) in accounting for variability and reflects the level of confidence. The level of confidence represents the level of protection afforded to facilities whose emissions are in line with the best performers, and consequently, the level of confidence is not arbitrary. For example, a 99% level of confidence means that a facility whose emissions are in line with the best performers has one chance in 100 of exceeding the floor limit. A prediction interval for a single future observation (or an average of several test observations) is an interval that will, with a specified degree of confidence, contain the next (or the average of some other pre-specified number) of randomly selected observation(s) from a population. In other words, the UPL estimates what the upper bound of future values will be, based upon present or past background samples taken. The UPL consequently represents the value at which we can expect the mean of future observations for the HAP or HAP surrogate emissions to fall within a specified level of confidence, based upon the measurements from an independent sample from the same population. The UPL approach encompasses all the data point-to-data point variability.

The form of the UPL equation differs depending upon the nature of the data set to which it is applied. To this end, the data sets were evaluated for each HAP and HAP surrogate to ascertain whether the data were distributed normally, log-normally, or skew-normally. Statistical tests of the kurtosis and skewness are then used to evaluate the normality assumption. The skewness statistic, S, characterizes the degree of asymmetry of a given data distribution, and is calculated according to the Fisher method as:

$$S = \frac{n}{(n-1)(n-2)} \sum_{i=1}^{n} \left(\frac{x_i - \overline{x}}{\sigma} \right)^3$$

Where n is the sample size, $\overline{x}$ is the mean of emissions data, and σ is the standard deviation (Doric et al. 2007). Normally distributed data have $S = 0$. An S that is greater (less) than 0 indicates that the data are asymmetrically distributed with a right (left) tail extending towards positive (negative) values. The standard error of the skewness statistic, SES, can be approximated by $SES = \sqrt{\frac{6n(n-1)}{(n-2)(n+1)(n+3)}}$, where n is the sample size. The kurtosis statistic, K, characterizes the degree of peakedness or flatness of a given data distribution in comparison to a normal distribution, and is calculated according to the Fisher method as:

$$K = \frac{\sum_{i=1}^{n}(x_i - \overline{x})^4}{(n-1)\sigma^4} - 3$$

for small data sets where $n = 3$ and as:

$$K = \frac{n(n+1)}{(n-1)(n-2)(n-3)} \sum_{i=1}^{n} \left(\frac{x_i - \overline{x}}{\sigma} \right)^4 - \frac{3(n-1)^2}{(n-2)(n-3)}$$

for larger data sets. Normally distributed data have $K = 3$. A K that is greater (less) than 3 indicates a relatively peaked (flat) distribution. For small data sets where $n = 3$, the standard error of the kurtosis statistic, SEK, can be approximated by $SEK = \sqrt{24/n}$, where n is the sample size. Otherwise, $SEK = \sqrt{\frac{24n(n^2-1)}{(n-2)(n+3)(n-3)(n+5)}}$.

For each data set to which the UPL was applied (i.e., the separate data sets for each HAP and HAP surrogate type applicable to the subcategory), the S and K statistics were calculated using both the reported test run values and the log-transformed reported test run values. For small data sets where $n = 3$, if $\left|\frac{S}{SES}\right|$ for the non-transformed data is less than $\left|\frac{S}{SES}\right|$ for the log-transformed data then the distribution is considered normal, otherwise it is consider log-normal. The skew-normal distribution can only be implemented for data where $n > 3$. Generally, if $\left|\frac{S}{SES}\right|$ or $\left|\frac{K}{SEK}\right|$ are greater than `qnorm(0.975, 0, 1)` (the 97.5^{th} quantile of the standard normal, which is approximately 1.96) the data are considered non-normal. Using these ratios, a series of decisions is used to determine the best distribution choice. First, each ratio of $\left|\frac{S}{SES}\right|$ and $\left|\frac{K}{SEK}\right|$ are calculated, and if they are less than 1.96 for untransformed or log-transformed, the data are tentatively assigned as normal. If both the skewness and kurtosis ratios came back normal for the log-transformed data and the untransformed data, then we check: if $\left|\frac{S}{SES}\right|$ is smaller for the untransformed data than the log-transformed data, the distribution is finalized as normal, if it is larger than the distribution is finalized as log-normal. For all other circumstances where both the skewness and kurtosis ratios for the untransformed data were normal, the final distribution choice is normal. Similarly, for all other circumstances where both the skewness and kurtosis ratios for the log-transformed data were normal, the final distribution is log-normal. All other circumstances are designated for the skew-normal distribution. The distribution selection is implemented via the function `distribution_type()` in `UPLforOAR` version 0.1.0, which is documented in Appendix C.

UPL Calculation for Normal Data

In the case of normally distributed data, the MACT floor limit is an UPL calculated as:

$$UPL = \overline{x} + t_{df,\ 0.99} \sqrt{\sigma^2 \left(\frac{1}{n} + \frac{1}{m} \right)}$$

Where $\overline{x}$ is the mean of the top performing sources calculated as $\overline{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$, n is the number of test runs, m is the number of future test runs, σ^2 is the variance calculate as $\sigma^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \overline{x})^2$, df is the degrees

of freedom calculated as $df = n - 1$, and $t_{df,\ 0.99}$ is the 99^{th} quantile of the t-distribution with df degrees of freedom. This UPL calculation was performed in `R` using the `Normal_UPL()` function from the `UPLforOAR` package version 0.1.0. See Appendix C for the function details.

UPL Calculation for Log-normal Data

In the case of log-normally distributed data, the MACT floor limit is an UPL calculated from a log-normal approximation described in Bhaumik and Gibbons, 2004 using the Gram-Charlier Type-A Series expansion.

$$UPL = e^{\hat{\mu}+\frac{\hat{\sigma}^2}{2}} + \frac{z_{\ 0.99}}{m}\sqrt{me^{2\hat{\mu}+\hat{\sigma}^2}(e^{\hat{\sigma}^2}-1) + m^2e^{2\hat{\mu}+\hat{\sigma}^2}\left(\frac{\hat{\sigma}^2}{n} + \frac{\hat{\sigma}^4}{2(n-1)}\right)}$$

Where n is the number of test runs, m is the number of future test runs, $\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} log(x_i)$ is the mean of the log-transformed data from the top performing sources, $\hat{\sigma}^2 = \frac{1}{n-1}\sum_{i=1}^{n}(log(x_i) - \hat{\mu})^2$ is the variance of the log-transformed data, and $z_{\ 0.99}$ is the 99^{th} percentile of the log-normal distribution estimated using the trapezoidal rule from equation 26 in Bhaumik and Gibbons 2004:

$$0.99 = \int_0^{z_{\ 0.99}} \left|\left(1 - \frac{\sqrt{\beta_{1z}}}{6}(3z - z^3) + \frac{(\beta_{2z-3})(3 - 6z^2 + z^4)}{24}\right)\phi(z)\right|$$

Where $\phi(z)$ is the standard normal and z is a sequence from -5 to 5 by intervals of 0.101. The coefficients are calculated following equation 25 as:

$$\beta_{2z} = \frac{e^{4\hat{\sigma}^2} + 2e^{3\hat{\sigma}^2} + 3e^{2\hat{\sigma}^2} - 3}{m(e^{\hat{\sigma}^2} - 1)^2} + 3\left(1 - \frac{1}{m}\right)$$

and:

$$\sqrt{\beta_{1z}} = \frac{\sqrt{e^{\hat{\sigma}^2} - 1}(e^{\hat{\sigma}^2} + 2)}{\sqrt{m}}$$

This UPL calculation was performed in `R` using the `Lognormal_UPL()` function from the `UPLforOAR` package version 0.1.0. See Appendix C for the function details.

UPL Calculation for Skew-normal Data

The same UPL equation as for normal data is used, but the t-statistic is substituted for a new t-statistic that is iteratively adjusted for the skewed distribution until it achieves 99% significance. To achieve this, an approximation of the incomplete beta function, is used:

$$I_{\ u_0}(a,\ b) \approx \frac{\Gamma(a,w)}{\Gamma(a)} + \frac{e^{-w}w^a}{\Gamma(a)}\left(\frac{a-1-w}{2b} + \frac{1}{(2b)^2}\left(\frac{a^3}{2} - \frac{5a^2}{3} + \frac{3a}{2} - \frac{1}{3} - w\left(\frac{3a^2}{2} - \frac{11a}{6} + \frac{1}{3}\right) + w^2\left(\frac{3a}{2} - \frac{1}{6}\right) - w^3\frac{1}{2}\right)\right)$$

Where u_0 and w are defined as:

$$u_0 = \frac{1}{1 + \frac{t^2_{new}}{n-1}}, \quad w = b\left(\frac{u_0}{1-u_0}\right)$$

Where n is the number of test runs and t_{new} is the iteratively adjusted t-statistic. Six expanded terms of the incomplete beta function are calculated with $(a_i,\ b_i)$ pairs:

$$b_1,\ ...,\ b_6 = \frac{1}{2},\ \frac{1}{2},\ \frac{1}{2},\ \frac{1}{2},\ 1,\ 1$$

and:

$$a_1,\ ...,\ a_6 = \frac{n-1}{2},\ \frac{n+1}{2},\ \frac{n+3}{2},\ \frac{n+5}{2},\ \frac{n-1}{2},\ \frac{n+1}{2}$$

These six incomplete beta function terms, $I_{u_0}(a_i,\ b_i)$, for $i\ \epsilon\ (1,\ 6)$, are then used in the subsequent calculation of the probability $P(t_{new})$ given the trial t-statistic t_{new} and actual skewness S, and kurtosis K:

$$P(t_{new}) = 1 - \frac{I_{u_0}(a_1,\ b_1)}{2} - S \times \left(I_{u_0}(a_5,\ b_5) \times \left(\frac{2n-1}{6\sqrt{2n\pi}} \right) - I_{u_0}(a_6,\ b_6) \times \left(\frac{n-1}{3\sqrt{2n\pi}} \right) \right) +$$
$$K \times \left(I_{u_0}(a_1,\ b_1) \times \left(\frac{n-1}{24} \right) - I_{u_0}(a_2,\ b_2) \times \left(\frac{(n-1)(n+2)}{12n} \right) + I_{u_0}(a_3,\ b_3) \times \left(\frac{(n+4)(n-1)}{24n} \right) \right) -$$
$$S^2 \times \left(I_{u_0}(a_1,\ b_1) \times \left(\frac{(n-1)(2n+5)}{72} \right) - I_{u_0}(a_2,\ b_2) \times \left(\frac{(n-1)(2n^2+5n+8)}{24n} \right) - \right.$$
$$\left. I_{u_0}(a_3,\ b_3) \times \left(\frac{(n-1)(2n^2+5n+12)}{24n} \right) - I_{u_0}(a_4,\ b_4) \times \left(\frac{(n-1)(2n^2+5n+12)}{72n} \right) \right)$$

Where S and K are the skewness and kurtosis estimates defined earlier.

First, the updated probability is calculated using the original t-statistic after accounting for skewness and kurtosis. If this $P(t)$ is within 0.0001 of 0.99, then the UPL is calculated with the original t-statistic as:

$$UPL = \overline{x} + t_{df,\ 0.99} \sqrt{\sigma^2 \left(\frac{1}{n} + \frac{1}{m} \right)}$$

Which is the same as the UPL for normally distributed data. If, however, the updated probability $P(t)$ is more than 0.0001 different than 0.99, then a list of 20,000 potential t_{new} are proposed in increasing or decreasing increments of 0.0001 (depending on whether $P(t)$ is less than or greater than 0.99). The first t_{new} such that $P(t_{new})$ is within 0.0001 of 0.99 is selected as the new t-statistic to use in the UPL calculation for skew-normal data:

$$UPL = \overline{x} + t_{new} \sqrt{\sigma^2 \left(\frac{1}{n} + \frac{1}{m} \right)}$$

This UPL calculation was performed in `R` using the `Skewed_UPL()` function from the `UPLforOAR` package version 0.1.0. See Appendix C for the function details.

Adjustment for Below Detection Level Emissions Data

To account for the effect of measurement imprecision associated with the data in the ICR databases (that include method detection level data), a protocol specified by the EPA was used. The procedure for determining a representative detection level (RDL) begins with identifying all of the available reported pollutant-specific method detection levels for the best performing units regardless of any subcategory (e.g., existing or new, fuel type, etc.). From that combined pool of data, we calculate the arithmetic mean value. By limiting the data set to those tests used to establish the floor or emissions limit (i.e., best performers), the EPA believes that the result is representative also of the best performing testing companies and laboratories. The EPA also believes that the outcome should minimize the effect of a test(s) with an inordinately high method detection

level (e.g., the sample volume was too small, the laboratory technique was insufficiently sensitive, or the procedure for determining the detection level was other than that specified). The EPA then calls the resulting mean of the method detection levels the RDL, and the Agency considers it characteristic of accepted source emissions measurement performance.

The second step in the process is to calculate three times the RDL to compare with the calculated floor or emissions limit. We use the multiplication factor of 3. For comparing to the floor, if three times the RDL were less than the calculated floor or emissions limit (e.g., calculated from the UPL), we would conclude that measurement variability was adequately addressed. The calculated floor or emissions limit would need no adjustment. If, on the other hand, the value equal to three times the RDL were greater than the UPL, we would conclude that the calculated floor or emissions limit does not account entirely for measurement variability. If indicated, we substituted the value equal to three times the RDL for the calculated MACT floor UPL emissions level.

Add any RDL content specific to your source category

Table 1: Here is an example of a table in case you need one

	Existing MACT floor (old units)	Existing MACT floor (new units)	New MACT floor
Subcategory 1	HAP 1, HAP 2	HAP 1, HAP 2	HAP 1, HAP 2, X, Y, Z
Subcategory 2	HAP 1	HAP 1	HAP 1, X, Y, Z
Subcategory 3	HAP 1, HAP 2, HAP 3	HAP 1, HAP 2, HAP 3	HAP 1, HAP 2, HAP 3, X, Y, Z
Subcategory 4	None	None	None
Subcategory 5	None	None	X, Y, Z

describe any data that was excluded here and why

Tables with the full data sets used for each existing source MACT floor analysis can be found listed in Appendix A. Tables with the full data sets used for each new source MACT floor analysis can be found listed in Appendix B.

The final data set for the *chemical X* existing source MACT floor for *subcategory A* included 24 test runs from 5 sources. The best performer for the new source calculation was A with an average emission of 0.01004 from 3 test runs.

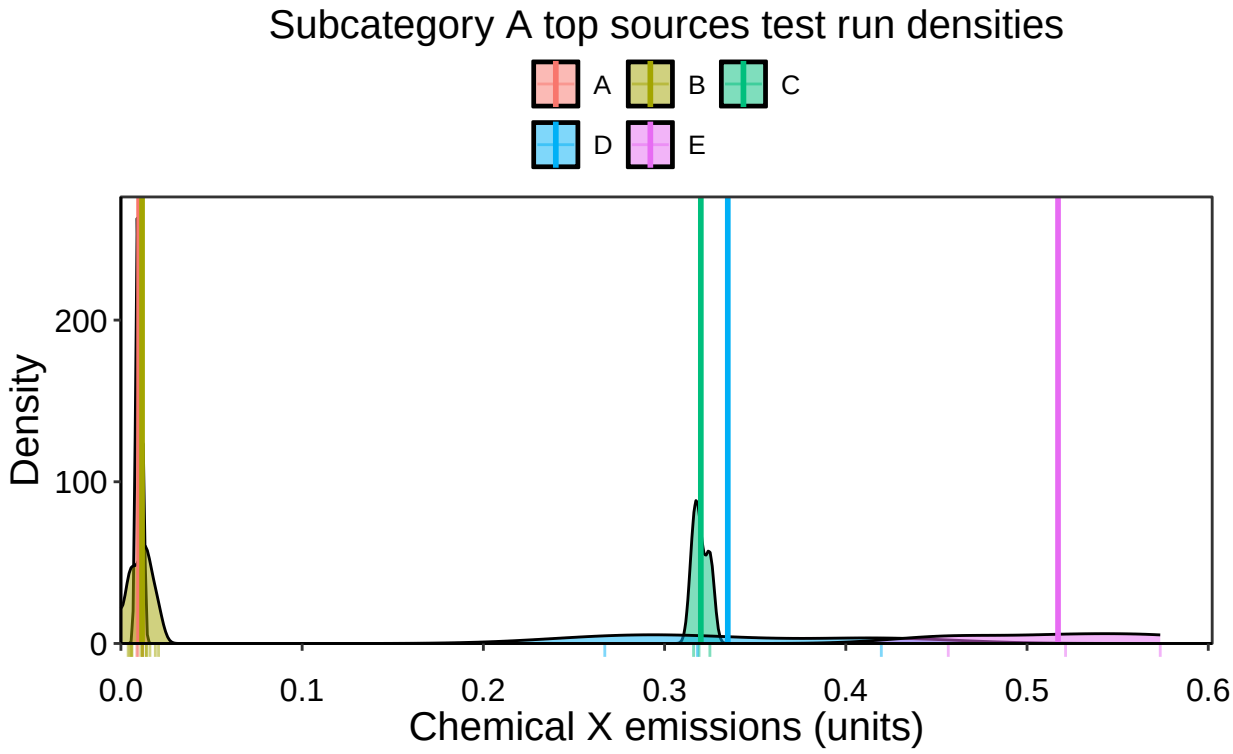

Figure 1: Observation density of emissions by source type. Source averages are indicated by vertical lines.

Figures with density plots of emission test run data were created using `ggplot2` in `R`. The density curves were generated using `obs_density()` in `UPLforOAR` version 0.1.0, using a `gamma` kernel type, a zero lower bound, a bandwidth of $\sigma n^{-2/5}$ where σ is the emission value standard deviation and n is the number of runs. For further details see the `R` code in Appendix C.

MACT Floor Analysis Results

Existing Source *Chemical X* in *Subcategory A*

The distribution that best represents the existing source data set for *chemical x* in *subcategory A* is the Lognormal distribution, with a corresponding UPL of 2.571 *(insert measurement units)*.

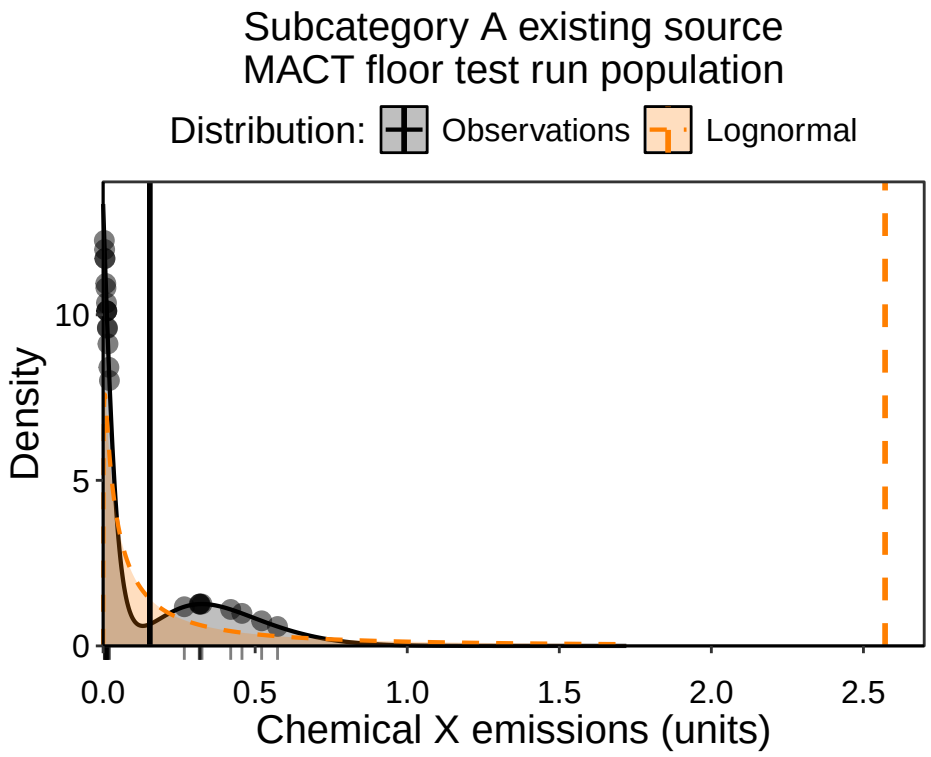

Figure 2: Observation density of chemical X emissions for the overall population of test runs for the existing source MACT floor for subcateogry A. The obseration data are indicated in black as points and a rug along the axis, with the observation density distribution as a black line. The fitted distribution that is the basis of the UPL estimate is colored orange. The average of the emission test runs is the vertical black line and the UPL result is the vertical orange line.

New Source *Chemical X* in *Subcategory A*

The distribution that best represents the new source data set for *chemical x* in *subcategory A* is the Lognormal distribution, with a corresponding UPL of 0.01484 *(insert measurement units)*.

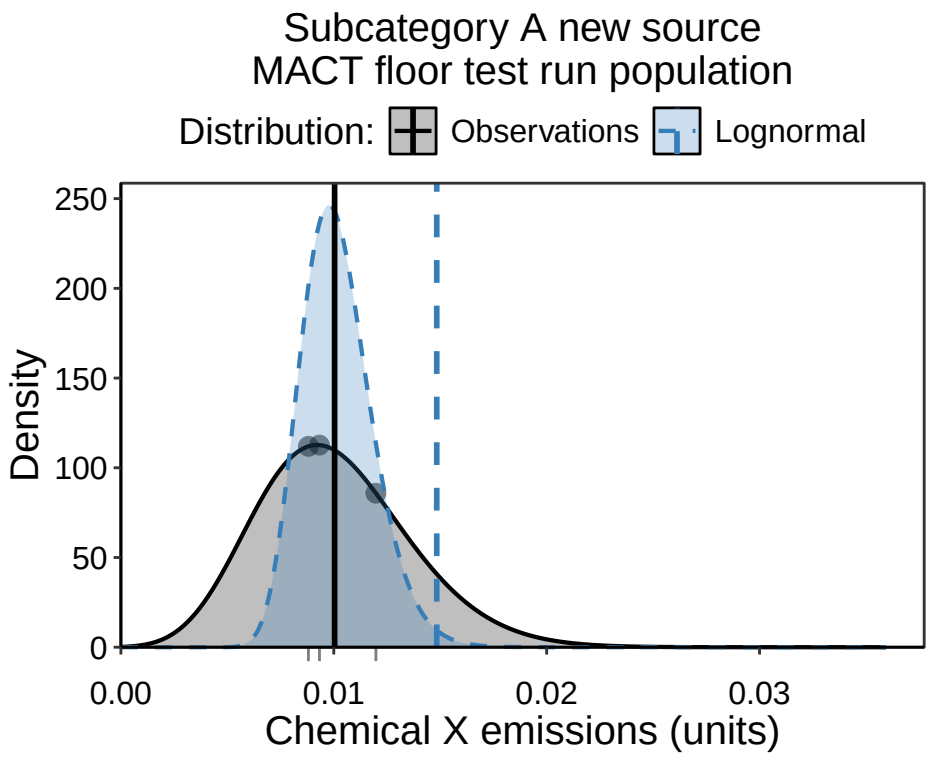

Figure 3: Observation density of chemical X emissions for the overall population of test runs for the new source MACT floor for subcategory A. The obseration data are indicated in black as points and a rug along the axis, with the observation density distribution as a black line. The fitted distribution that is the basis of the UPL estimate is colored blue. The average of the emission test runs is the vertical black line and the UPL result is the vertical blue line.

UPL Calculation Summary

The table below includes all UPL calculations for Example Industry existing and new source MACT floor analyses using a significance of 99% and 3 future test runs.

Table 2: Upper Predictive Limit (UPL) results for existing and new source MACT floors in (measurement units)

HAP	Subcategory	Existing source UPL	New source UPL
Chemical X	Type A	2.571	0.01484

References

Bhaumik, D.K. and Gibbons, R.D. 2004. An Upper Prediction Limit for the Arithmetic Mean of a Lognormal Random Variable. Technometrics, 46(2):239-248.

Dragan Doric, et al. 2007. On measuring skewness and kurtosis. Springer Science + Buisness Media B. V. 2007.

Renault O. & O. Scaillet. On the way to recovery: A parametric bias free estimation of recovery rate densities. 2004. Journal of Banking and Finance. 28:12 p. 2915-2931.

Appendix A: Existing Source Data Tables

Example Industry existing source MACT floor analysis data set for HAP *chemical X* in *subcategory A*:

Table 3: Top performering source emissions data used in existing source UPL calculations.

emissions	sources	ln_emiss
0.008808290	A	-4.7320620
0.011979167	A	-4.4245862
0.009326425	A	-4.6749035
0.011052632	B	-4.5050867
0.020769870	B	-3.8742519
0.014084507	B	-4.2626799
0.012000000	B	-4.4228486
0.006000000	B	-5.1159958
0.004000000	B	-5.5214609
0.019000000	B	-3.9633163
0.014000000	B	-4.2686979
0.016000000	B	-4.1351666
0.012000000	B	-4.4228486
0.006000000	B	-5.1159958
0.005000000	B	-5.2983174
0.316000000	C	-1.1520131
0.325000000	C	-1.1239301
0.319000000	C	-1.1425642
0.419587629	D	-0.8684829
0.267010309	D	-1.3204680
0.318041237	D	-1.1455742
0.521265823	E	-0.6514952
0.456532663	E	-0.7840950
0.573500000	E	-0.5559973

Appendix B: New Source Data Tables

Example Industry new source MACT floor analysis data set for HAP *chemical X* in *subcategory A*:

Table 4: Best performering source emissions data used in new source UPL calculations.

emissions	sources	ln_emiss
0.008808290	A	-4.732062
0.011979167	A	-4.424586
0.009326425	A	-4.674904

Appendix C: R Function Code

Disclaimer

The United States Environmental Protection Agency (EPA) GitHub project code is provided on an "as is" basis and the user assumes responsibility for its use. EPA has relinquished control of the information and no longer has responsibility to protect the integrity , confidentiality, or availability of the information. Any reference to specific commercial products, processes, or services by service mark, trademark, manufacturer, or otherwise, does not constitute or imply their endorsement, recommendation or favoring by EPA. The EPA seal and logo shall not be used in any manner to imply endorsement of any commercial product or activity by EPA or the United States Government.

MACT_existing()

```
#> function (data, emissions, sources, CAA_section = 112)
#> {
#>     data_temp = tibble::tibble(emissions = data[[emissions]],
#>         sources = data[[sources]])
#>     if (!is.numeric(data_temp$emissions)) {
#>         stop("Emissions must be numeric vector")
#>     }
#>     if ((!is.character(data_temp$sources)) & (!is.factor(data_temp$sources))) {
#>         stop("Sources must be a character or factor vector")
#>     }
#>     if (any(data_temp$emissions < 0)) {
#>         warning("emissions data contain negative values")
#>     }
#>     if (any(data_temp$emissions == 0)) {
#>         warning("emissions data contain zero values and have been removed")
#>         data_temp = subset(data_temp, data_temp$emissions !=
#>             0)
#>     }
#>     dat_means = dplyr::summarize(data_temp, means = mean(emissions),
#>         .by = "sources")
#>     n_sources = length(unique(data_temp$sources))
#>     if (CAA_section == 129) {
#>         n_topsources = ceiling(0.12 * n_sources)
#>         set.seed(1)
#>         source_index = sample.int(n_sources, n_topsources)
#>         dat_top = data_temp[source_index, ]
#>     }
#>     else if (CAA_section == 112) {
#>         if (n_sources >= 30) {
#>             n_topsources = ceiling(0.12 * n_sources)
#>             top_list = dat_means[order(dat_means$means, decreasing = F),
#>                 ]
#>             top_list = top_list$sources[1:n_topsources]
#>             dat_top = subset(data_temp, data_temp$sources %in%
#>                 top_list)
#>         }
#>         else if (n_sources < 30) {
#>             n_topsources = 5
#>             top_list = dat_means[order(dat_means$means, decreasing = F),
#>                 ]
#>             top_list = top_list$sources[1:n_topsources]
```

```
#>             dat_top = subset(data_temp, data_temp$sources %in%
#>                 top_list)
#>         }
#>     }
#>     dat_topmeans = dplyr::summarize(dat_top, means = mean(emissions),
#>         .by = "sources", counts = dplyr::n())
#>     dat_topmeans$sources = as.factor(dat_topmeans$sources)
#>     dat_topmeans$sources = forcats::fct_reorder(dat_topmeans$sources,
#>         dat_topmeans$means, .desc = FALSE)
#>     dat_top$sources = factor(dat_top$sources, levels = levels(dat_topmeans$sources))
#>     dat_topmeans = dplyr::arrange(dat_topmeans, means)
#>     dat_top = dplyr::arrange(dat_top, sources)
#>     dat_top$sources = droplevels(dat_top$sources)
#>     if (!is.character(emissions)) {
#>         emissions = colnames(data)[emissions]
#>     }
#>     if (!is.character(sources)) {
#>         sources = colnames(data)[sources]
#>     }
#>     names(dat_top) = c(emissions, sources)
#>     return(dat_top)
#> }
#> <bytecode: 0x0000016f37b34518>
#> <environment: namespace:UPLforOAR>
```

MACT_new()

```
#> function (data, emissions, sources)
#> {
#>     data_temp = tibble::tibble(emissions = data[[emissions]],
#>         sources = data[[sources]])
#>     if (!is.numeric(data_temp$emissions)) {
#>         stop("Emissions must be numeric vector")
#>     }
#>     if ((!is.character(data_temp$sources)) & (!is.factor(data_temp$sources))) {
#>         stop("Sources must be a character or factor vector")
#>     }
#>     if (any(data_temp$emissions < 0)) {
#>         warning("emissions data contain negative values")
#>     }
#>     if (any(data_temp$emissions == 0)) {
#>         warning("emissions data contain zero values and have been removed")
#>         data_temp = subset(data_temp, data_temp$emissions !=
#>             0)
#>     }
#>     dat_means = dplyr::summarize(data_temp, means = mean(emissions),
#>         .by = "sources")
#>     top_list = dat_means[order(dat_means$means, decreasing = F),
#>         ]
#>     top_source = top_list$sources[1]
#>     dat_top = subset(data_temp, data_temp$sources == top_source)
#>     dat_top$sources = as.factor(dat_top$sources)
#>     dat_top$sources = droplevels(dat_top$sources)
#>     if (!is.character(emissions)) {
```

```
#>         emissions = colnames(data)[emissions]
#>     }
#>     if (!is.character(sources)) {
#>         sources = colnames(data)[sources]
#>     }
#>     names(dat_top) = c(emissions, sources)
#>     return(dat_top)
#> }
#> <bytecode: 0x0000016f387803e0>
#> <environment: namespace:UPLforOAR>
```

distribution_type()

```
#> function (data, emissions)
#> {
#>     data_temp = tibble::tibble(emissions = data[[emissions]])
#>     data_temp$ln_emiss = log(data_temp$emissions)
#>     data_temp$ln_emiss = replace(data_temp$ln_emiss, !is.finite(data_temp$ln_emiss),
#>         NA)
#>     sigma = stats::sd(data_temp$emissions)
#>     mean_ln = mean(data_temp$ln_emiss, na.rm = TRUE)
#>     sigma_ln = stats::sd(data_temp$ln_emiss, na.rm = TRUE)
#>     if ((sigma == 0) | (sigma_ln == 0)) {
#>         stop("Cannot perform UPL calculation on data with 0 variance")
#>     }
#>     n = length(data_temp$emissions)
#>     if (n < 3) {
#>         stop("Need at least 3 observations for UPL calculation")
#>     }
#>     emission_mean = mean(data_temp$emissions)
#>     S = n/((n - 1) * (n - 2)) * sum(((data_temp$emissions - emission_mean)/sigma)^3)
#>     S_ln = n/((n - 1) * (n - 2)) * sum(((data_temp$ln_emiss -
#>         mean_ln)/sigma_ln)^3, na.rm = T)
#>     SES = sqrt((6 * n * (n - 1))/((n - 2) * (n + 1) * (n + 3)))
#>     S_SES = abs(S/SES)
#>     S_SES_ln = abs(S_ln/SES)
#>     if (n == 3) {
#>         K = sum((data_temp$emissions - emission_mean)^4)/((n -
#>             1) * (sigma)^4) - 3
#>         K_ln = sum((data_temp$ln_emiss - mean_ln)^4, na.rm = T)/((n -
#>             1) * (sigma_ln)^4) - 3
#>         SEK = sqrt(24/n)
#>         if (S_SES < S_SES_ln) {
#>             distr_choice = "Normal"
#>         }
#>         else {
#>             distr_choice = "Lognormal"
#>         }
#>     }
#>     else if (n > 3) {
#>         K = (n * (n + 1))/((n - 1) * (n - 2) * (n - 3)) * sum(((data_temp$emissions -
#>             emission_mean)/sigma)^4) - (3 * (n - 1)^2)/((n -
#>             2) * (n - 3))
#>         K_ln = (n * (n + 1))/((n - 1) * (n - 2) * (n - 3)) *
```

```
#>             sum(((data_temp$ln_emiss - mean_ln)/sigma_ln)^4,
#>                 na.rm = T) - (3 * (n - 1)^2)/((n - 2) * (n -
#>             3))
#>         SEK = sqrt(24 * n * (n^2 - 1)/((n - 2) * (n + 3) * (n -
#>             3) * (n + 5)))
#>         S_SEK = abs(K/SEK)
#>         S_SEK_ln = abs(K_ln/SEK)
#>         norm_zscore = stats::qnorm(0.975)
#>         if (S_SES > norm_zscore) {
#>             raw_distr1 = "Non-normal"
#>         }
#>         else {
#>             raw_distr1 = "Normal"
#>         }
#>         if (S_SEK > norm_zscore) {
#>             raw_distr2 = "Non-normal"
#>         }
#>         else {
#>             raw_distr2 = "Normal"
#>         }
#>         if (S_SES_ln > norm_zscore) {
#>             ln_distr1 = "Non-normal"
#>         }
#>         else {
#>             ln_distr1 = "Normal"
#>         }
#>         if (S_SEK_ln > norm_zscore) {
#>             ln_distr2 = "Non-normal"
#>         }
#>         else {
#>             ln_distr2 = "Normal"
#>         }
#>         if ((raw_distr1 == "Normal") & (raw_distr2 == "Normal")) {
#>             raw_distr = "Normal"
#>         }
#>         else {
#>             raw_distr = "Non-normal"
#>         }
#>         if ((ln_distr1 == "Normal") & (ln_distr2 == "Normal")) {
#>             ln_distr = "Normal"
#>         }
#>         else {
#>             ln_distr = "Non-normal"
#>         }
#>         if ((ln_distr == "Normal") & (raw_distr == "Normal")) {
#>             if (S_SES < S_SES_ln) {
#>                 distr_choice = "Normal"
#>             }
#>             else {
#>                 distr_choice = "Lognormal"
#>             }
#>         }
#>         else if (raw_distr == "Normal") {
#>             distr_choice = "Normal"
```

```
#>         }
#>         else if (ln_distr == "Normal") {
#>             distr_choice = "Lognormal"
#>         }
#>         else {
#>             distr_choice = "Skewed"
#>         }
#>     }
#>     return(distr_choice)
#> }
#> <bytecode: 0x0000016f3951a9e8>
#> <environment: namespace:UPLforOAR>
```

Normal_UPL()

```
#> function (data, emissions, future_runs = 3, significance = 0.99)
#> {
#>     future_runs = as.integer(future_runs)
#>     if (!is.integer(future_runs)) {
#>         stop("future_runs must be a positive integer")
#>     }
#>     if (future_runs < 1) {
#>         stop("future_runs must be a positive integer")
#>     }
#>     if (significance >= 1) {
#>         stop("significance must be greater then 0 and less than 1")
#>     }
#>     if (significance <= 0) {
#>         stop("significance must be greater then 0 and less than 1")
#>     }
#>     data_temp = tibble::tibble(emissions = data[[emissions]])
#>     n = length(data_temp$emissions)
#>     if (n < 3) {
#>         stop("Need at least 3 observations for UPL calculation")
#>     }
#>     df = n - 1
#>     tscore = stats::qt(significance, df)
#>     emission_mean = mean(data_temp$emissions)
#>     var.s = sum((data_temp$emissions - emission_mean)^2) * (1/(n -
#>         1))
#>     if (var.s == 0) {
#>         stop("Cannot perform UPL calculation on data with 0 variance")
#>     }
#>     UPL_normal = emission_mean + tscore * sqrt(var.s * (1/n +
#>         1/future_runs))
#>     return(UPL_normal)
#> }
#> <bytecode: 0x0000016f31ba47b8>
#> <environment: namespace:UPLforOAR>
```

Lognormal_UPL()

```
#> function (data, emissions, future_runs = 3, significance = 0.99)
#> {
#>     future_runs = as.integer(future_runs)
```

```
#>     if (!is.integer(future_runs)) {
#>         stop("future_runs must be a positive integer")
#>     }
#>     if (future_runs < 1) {
#>         stop("future_runs must be a positive integer")
#>     }
#>     if (significance >= 1) {
#>         stop("significance must be greater then 0 and less than 1")
#>     }
#>     if (significance <= 0) {
#>         stop("significance must be greater then 0 and less than 1")
#>     }
#>     data_temp = tibble::tibble(emissions = data[[emissions]])
#>     data_temp$ln_emiss = log(data_temp$emissions)
#>     data_temp$ln_emiss = replace(data_temp$ln_emiss, !is.finite(data_temp$ln_emiss),
#>         NA)
#>     n = length(data_temp$emissions)
#>     if (n < 3) {
#>         stop("Need at least 3 observations for UPL calculation")
#>     }
#>     mean_log = mean(data_temp$ln_emiss, na.rm = TRUE)
#>     sd_log = stats::sd(data_temp$ln_emiss, na.rm = TRUE)
#>     if (sd_log == 0) {
#>         stop("Cannot perform UPL calculation on data with 0 variance")
#>     }
#>     beta2 = (exp(4 * sd_log^2) + 2 * exp(3 * sd_log^2) + 3 *
#>         exp(2 * sd_log^2) - 3)/(future_runs * (exp(sd_log^2) -
#>         1)^2) + 3 * (1 - 1/future_runs)
#>     beta1 = sqrt(exp(sd_log^2) - 1) * (exp(sd_log^2) + 2)/sqrt(future_runs)
#>     zvals = seq(-5, 5, by = 0.0101)
#>     Fgzs = abs((1 - (beta1/6) * (3 * zvals - zvals^3) + (beta2 -
#>         3) * (3 - 6 * zvals^2 + zvals^4)/24) * stats::dnorm(zvals))
#>     Fg_cdf = cumsum(Fgzs/sum(Fgzs))
#>     zscore1 = zvals[Fg_cdf > significance][1]
#>     mean_convert = exp(mean_log + (sd_log^2)/2)
#>     part2 = sqrt(future_runs * exp(2 * mean_log + sd_log^2) *
#>         (exp(sd_log^2) - 1) + future_runs^2 * exp(2 * mean_log +
#>         sd_log^2) * (((sd_log^2)/n) + (sd_log^4)/(2 * (n - 1))))
#>     UPL_lognormal = mean_convert + (zscore1/future_runs) * part2
#>     return(UPL_lognormal)
#> }
#> <bytecode: 0x0000016f39657c50>
#> <environment: namespace:UPLforOAR>
```

Skewed_UPL()

```
#> function (data, emissions, future_runs = 3, significance = 0.99)
#> {
#>     data_temp = tibble::tibble(emissions = data[[emissions]])
#>     n = length(data_temp$emissions)
#>     if (n < 3) {
#>         stop("Need at least 3 observations for UPL calculation")
#>     }
#>     if (n > 341) {
```

```
#>         warning("slower calculations due to requirement for high precision floating point")
#>     }
#>     future_runs = as.integer(future_runs)
#>     emission_mean = mean(data_temp$emissions, na.rm = T)
#>     sigma = stats::sd(data_temp$emissions, na.rm = T)
#>     if (sigma == 0) {
#>         stop("Cannot perform UPL calculation on data with 0 variance")
#>     }
#>     if (!is.integer(future_runs)) {
#>         stop("future_runs must be a positive integer")
#>     }
#>     if (future_runs < 1) {
#>         stop("future_runs must be a positive integer")
#>     }
#>     if (significance >= 1) {
#>         stop("significance must be greater then 0 and less than 1")
#>     }
#>     if (significance <= 0) {
#>         stop("significance must be greater then 0 and less than 1")
#>     }
#>     if (n <= 3) {
#>         Skewed_UPL = NA
#>         warning("data must have more than 3 observations for skew UPL method")
#>     }
#>     else {
#>         K = (n * (n + 1))/((n - 1) * (n - 2) * (n - 3)) * sum(((data_temp$emissions -
#>             emission_mean)/sigma)^4) - (3 * (n - 1)^2)/((n -
#>             2) * (n - 3))
#>         S = n/((n - 1) * (n - 2)) * sum(((data_temp$emissions -
#>             emission_mean)/sigma)^3)
#>         df = n - 1
#>         var.s = sum((data_temp$emissions - mean(data_temp$emissions))^2) *
#>             (1/(n - 1))
#>         tscore = stats::qt(significance, df)
#>         u0 = 1/(1 + (tscore^2/(n - 1)))
#>         b = c(0.5, 0.5, 0.5, 0.5, 1, 1)
#>         w = b * (u0/(1 - u0))
#>         a = c((n - 1)/2, (n + 1)/2, (n + 3)/2, (n + 5)/2, (n -
#>             1)/2, (n + 1)/2)
#>         I_term = c()
#>         for (i in 1:6) {
#>             c1 = stats::pgamma(w[i], shape = a[i], rate = 1)
#>             c4 = (a[i] - 1 - w[i])/(2 * b[i])
#>             c5 = (a[i]^3/2 - 5 * a[i]^2/3 + 3 * a[i]/2 - 1/3)
#>             c6 = w[i] * (3 * a[i]^2/2 - 11 * a[i]/6 + 1/3)
#>             c7 = w[i]^2 * (3 * a[i]/2 - 1/6)
#>             if (n > 341) {
#>                 c2 = Rmpfr::igamma(a[i], 0)
#>                 c3 = ((exp(-w[i]) * w[i]^Rmpfr::mpfr(a[i], 128))/Rmpfr::igamma(a[i],
#>                   0))
#>                 I_term[i] = Rmpfr::asNumeric(c1/c2 + c3 * (c4 +
#>                   (1/(2 * b[i])^2) * (c5 - c6 + c7 - w[i]^3/2)))
#>             }
#>             else {
```

```
#>                 c2 = gamma(a[i])
#>                 c3 = ((exp(-w[i]) * w[i]^a[i])/gamma(a[i]))
#>                 I_term[i] = (c1/c2 + c3 * (c4 + (1/(2 * b[i])^2) *
#>                   (c5 - c6 + c7 - w[i]^3/2)))
#>             }
#>         }
#>         coeff1 = (2 * n - 1) * I_term[[5]]/(6 * sqrt(2 * n *
#>             pi)) - (n - 1) * I_term[[6]]/(3 * sqrt(2 * n * pi))
#>         coeff2 = (n - 1) * I_term[1]/24 - (n - 1) * (n + 2) *
#>             I_term[2]/(12 * n) + (n + 4) * (n - 1) * I_term[3]/(24 *
#>             n)
#>         coeff3 = (n - 1) * (2 * n + 5) * I_term[1]/72 - (n -
#>             1) * (2 * n^2 + 5 * n + 8) * I_term[2]/(24 * n) +
#>             (n - 1) * (2 * n^2 + 5 * n + 12) * I_term[3]/(24 *
#>                 n) - (n - 1) * (2 * n^2 + 5 * n + 12) * I_term[4]/(72 *
#>             n)
#>         current_prob = 1 - (I_term[1]/2 + S * coeff1 - K * coeff2 +
#>             S^2 * coeff3)
#>         if (abs(current_prob - significance) < 1e-04) {
#>             Skewed_UPL = emission_mean + tscore * sqrt(var.s *
#>                 (1/n + 1/future_runs))
#>         }
#>         else {
#>             tstat_list = seq(from = (tscore - 1), to = (tscore +
#>                 1), by = 1e-04)
#>             new_prob = c()
#>             for (t in 1:length(tstat_list)) {
#>                 u0 = 1/(1 + (tstat_list[t]^2/(n - 1)))
#>                 b = c(0.5, 0.5, 0.5, 0.5, 1, 1)
#>                 w = b * (u0/(1 - u0))
#>                 a = c((n - 1)/2, (n + 1)/2, (n + 3)/2, (n + 5)/2,
#>                   (n - 1)/2, (n + 1)/2)
#>                 I_term = c()
#>                 for (i in 1:6) {
#>                   c1 = stats::pgamma(w[i], shape = a[i], rate = 1)
#>                   c4 = (a[i] - 1 - w[i])/(2 * b[i])
#>                   c5 = (a[i]^3/2 - 5 * a[i]^2/3 + 3 * a[i]/2 -
#>                     1/3)
#>                   c6 = w[i] * (3 * a[i]^2/2 - 11 * a[i]/6 + 1/3)
#>                   c7 = w[i]^2 * (3 * a[i]/2 - 1/6)
#>                   if (n > 341) {
#>                     c2 = Rmpfr::igamma(a[i], 0)
#>                     c3 = ((exp(-w[i]) * w[i]^Rmpfr::mpfr(a[i],
#>                       128))/Rmpfr::igamma(a[i], 0))
#>                     I_term[i] = Rmpfr::asNumeric(c1/c2 + c3 *
#>                       (c4 + (1/(2 * b[i])^2) * (c5 - c6 + c7 -
#>                         w[i]^3/2)))
#>                   }
#>                   else {
#>                     c2 = gamma(a[i])
#>                     c3 = ((exp(-w[i]) * w[i]^a[i])/gamma(a[i]))
#>                     I_term[i] = (c1/c2 + c3 * (c4 + (1/(2 * b[i])^2) *
#>                       (c5 - c6 + c7 - w[i]^3/2)))
#>                   }
```

```
#>                 }
#>                 coeff1 = (2 * n - 1) * I_term[5]/(6 * sqrt(2 *
#>                   n * pi)) - (n - 1) * I_term[6]/(3 * sqrt(2 *
#>                   n * pi))
#>                 coeff2 = (n - 1) * I_term[1]/24 - (n - 1) * (n +
#>                   2) * I_term[2]/(12 * n) + (n + 4) * (n - 1) *
#>                   I_term[3]/(24 * n)
#>                 coeff3 = (n - 1) * (2 * n + 5) * I_term[1]/72 -
#>                   (n - 1) * (2 * n^2 + 5 * n + 8) * I_term[2]/(24 *
#>                     n) + (n - 1) * (2 * n^2 + 5 * n + 12) * I_term[3]/(24 *
#>                   n) - (n - 1) * (2 * n^2 + 5 * n + 12) * I_term[4]/(72 *
#>                   n)
#>                 new_prob[t] = 1 - (I_term[1]/2 + S * coeff1 -
#>                   K * coeff2 + S^2 * coeff3)
#>             }
#>             good_tscore = which(abs(new_prob - significance) <
#>                 1e-04)
#>             if (length(good_tscore) > 0) {
#>                 new_tscore = tstat_list[good_tscore[which.min(abs(good_tscore -
#>                   which(tstat_list == tscore)))]]
#>                 Skewed_UPL = mean(data_temp$emissions) + new_tscore *
#>                   sqrt(var.s * (1/n + 1/future_runs))
#>             }
#>             else if (between(significance, min(new_prob), max(new_prob))) {
#>                 closest_t = tstat_list[which.min(abs(new_prob -
#>                   significance))]
#>                 reset_t1 = tstat_list[which.min(abs(new_prob -
#>                   significance)) - 1]
#>                 reset_t2 = tstat_list[which.min(abs(new_prob -
#>                   significance)) + 1]
#>                 tstat_list = seq(from = reset_t1, to = reset_t2,
#>                   length.out = 5000)
#>                 new_prob = c()
#>                 for (t in 1:length(tstat_list)) {
#>                   u0 = 1/(1 + (tstat_list[t]^2/(n - 1)))
#>                   b = c(0.5, 0.5, 0.5, 0.5, 1, 1)
#>                   w = b * (u0/(1 - u0))
#>                   a = c((n - 1)/2, (n + 1)/2, (n + 3)/2, (n +
#>                     5)/2, (n - 1)/2, (n + 1)/2)
#>                   I_term = c()
#>                   for (i in 1:6) {
#>                     c1 = stats::pgamma(w[i], shape = a[i], rate = 1)
#>                     c4 = (a[i] - 1 - w[i])/(2 * b[i])
#>                     c5 = (a[i]^3/2 - 5 * a[i]^2/3 + 3 * a[i]/2 -
#>                       1/3)
#>                     c6 = w[i] * (3 * a[i]^2/2 - 11 * a[i]/6 +
#>                       1/3)
#>                     c7 = w[i]^2 * (3 * a[i]/2 - 1/6)
#>                     if (n > 341) {
#>                       c2 = Rmpfr::igamma(a[i], 0)
#>                       c3 = ((exp(-w[i]) * w[i]^Rmpfr::mpfr(a[i],
#>                         128))/Rmpfr::igamma(a[i], 0))
#>                       I_term[i] = Rmpfr::asNumeric(c1/c2 + c3 *
#>                         (c4 + (1/(2 * b[i]))^2) * (c5 - c6 + c7 -
```

```
#>                         w[i]^3/2)))
#>                     }
#>                     else {
#>                       c2 = gamma(a[i])
#>                       c3 = ((exp(-w[i]) * w[i]^a[i])/gamma(a[i]))
#>                       I_term[i] = (c1/c2 + c3 * (c4 + (1/(2 *
#>                         b[i])^2) * (c5 - c6 + c7 - w[i]^3/2)))
#>                     }
#>                   }
#>                   coeff1 = (2 * n - 1) * I_term[5]/(6 * sqrt(2 *
#>                     n * pi)) - (n - 1) * I_term[6]/(3 * sqrt(2 *
#>                     n * pi))
#>                   coeff2 = (n - 1) * I_term[1]/24 - (n - 1) *
#>                     (n + 2) * I_term[2]/(12 * n) + (n + 4) *
#>                     (n - 1) * I_term[3]/(24 * n)
#>                   coeff3 = (n - 1) * (2 * n + 5) * I_term[1]/72 -
#>                     (n - 1) * (2 * n^2 + 5 * n + 8) * I_term[2]/(24 *
#>                       n) + (n - 1) * (2 * n^2 + 5 * n + 12) *
#>                     I_term[3]/(24 * n) - (n - 1) * (2 * n^2 +
#>                     5 * n + 12) * I_term[4]/(72 * n)
#>                   new_prob[t] = 1 - (I_term[1]/2 + S * coeff1 -
#>                     K * coeff2 + S^2 * coeff3)
#>                 }
#>                 good_tscore = which(abs(new_prob - significance) <
#>                   1e-04)
#>                 new_tscore = tstat_list[good_tscore[which.min(abs(good_tscore -
#>                   which.min(abs(tstat_list - closest_t))))]]
#>             }
#>             else {
#>                 tstat_list = seq(from = (tscore - 3), to = (tscore +
#>                   3), by = 1e-04)
#>                 new_prob = c()
#>                 for (t in 1:length(tstat_list)) {
#>                   u0 = 1/(1 + (tstat_list[t]^2/(n - 1)))
#>                   b = c(0.5, 0.5, 0.5, 0.5, 1, 1)
#>                   w = b * (u0/(1 - u0))
#>                   a = c((n - 1)/2, (n + 1)/2, (n + 3)/2, (n +
#>                     5)/2, (n - 1)/2, (n + 1)/2)
#>                   I_term = c()
#>                   for (i in 1:6) {
#>                     c1 = stats::pgamma(w[i], shape = a[i], rate = 1)
#>                     c4 = (a[i] - 1 - w[i])/(2 * b[i])
#>                     c5 = (a[i]^3/2 - 5 * a[i]^2/3 + 3 * a[i]/2 -
#>                       1/3)
#>                     c6 = w[i] * (3 * a[i]^2/2 - 11 * a[i]/6 +
#>                       1/3)
#>                     c7 = w[i]^2 * (3 * a[i]/2 - 1/6)
#>                     if (n > 341) {
#>                       c2 = Rmpfr::igamma(a[i], 0)
#>                       c3 = ((exp(-w[i]) * w[i]^Rmpfr::mpfr(a[i],
#>                         128))/Rmpfr::igamma(a[i], 0))
#>                       I_term[i] = Rmpfr::asNumeric(c1/c2 + c3 *
#>                         (c4 + (1/(2 * b[i])^2) * (c5 - c6 + c7 -
#>                           w[i]^3/2)))
```

```
#>                     }
#>                     else {
#>                       c2 = gamma(a[i])
#>                       c3 = ((exp(-w[i]) * w[i]^a[i])/gamma(a[i]))
#>                       I_term[i] = (c1/c2 + c3 * (c4 + (1/(2 *
#>                         b[i])^2) * (c5 - c6 + c7 - w[i]^3/2)))
#>                     }
#>                   }
#>                   coeff1 = (2 * n - 1) * I_term[5]/(6 * sqrt(2 *
#>                     n * pi)) - (n - 1) * I_term[6]/(3 * sqrt(2 *
#>                     n * pi))
#>                   coeff2 = (n - 1) * I_term[1]/24 - (n - 1) *
#>                     (n + 2) * I_term[2]/(12 * n) + (n + 4) *
#>                     (n - 1) * I_term[3]/(24 * n)
#>                   coeff3 = (n - 1) * (2 * n + 5) * I_term[1]/72 -
#>                     (n - 1) * (2 * n^2 + 5 * n + 8) * I_term[2]/(24 *
#>                       n) + (n - 1) * (2 * n^2 + 5 * n + 12) *
#>                     I_term[3]/(24 * n) - (n - 1) * (2 * n^2 +
#>                     5 * n + 12) * I_term[4]/(72 * n)
#>                   new_prob[t] = 1 - (I_term[1]/2 + S * coeff1 -
#>                     K * coeff2 + S^2 * coeff3)
#>                 }
#>                 good_tscore = which(abs(new_prob - significance) <
#>                   1e-04)
#>                 if (length(good_tscore) > 0) {
#>                   new_tscore = tstat_list[good_tscore[which.min(abs(good_tscore -
#>                     which(tstat_list == tscore)))]]
#>                   Skewed_UPL = mean(data_temp$emissions) + new_tscore *
#>                     sqrt(var.s * (1/n + 1/future_runs))
#>                 }
#>                 else if (between(significance, min(new_prob),
#>                   max(new_prob))) {
#>                   closest_t = tstat_list[which.min(abs(new_prob -
#>                     significance))]
#>                   reset_t1 = tstat_list[which.min(abs(new_prob -
#>                     significance)) - 1]
#>                   reset_t2 = tstat_list[which.min(abs(new_prob -
#>                     significance)) + 1]
#>                   tstat_list = seq(from = reset_t1, to = reset_t2,
#>                     length.out = 5000)
#>                   new_prob = c()
#>                   for (t in 1:length(tstat_list)) {
#>                     u0 = 1/(1 + (tstat_list[t]^2/(n - 1)))
#>                     b = c(0.5, 0.5, 0.5, 0.5, 1, 1)
#>                     w = b * (u0/(1 - u0))
#>                     a = c((n - 1)/2, (n + 1)/2, (n + 3)/2, (n +
#>                       5)/2, (n - 1)/2, (n + 1)/2)
#>                     I_term = c()
#>                     for (i in 1:6) {
#>                       c1 = stats::pgamma(w[i], shape = a[i],
#>                         rate = 1)
#>                       c4 = (a[i] - 1 - w[i])/(2 * b[i])
#>                       c5 = (a[i]^3/2 - 5 * a[i]^2/3 + 3 * a[i]/2 -
#>                         1/3)
```

```
#>                       c6 = w[i] * (3 * a[i]^2/2 - 11 * a[i]/6 +
#>                         1/3)
#>                       c7 = w[i]^2 * (3 * a[i]/2 - 1/6)
#>                       if (n > 341) {
#>                         c2 = Rmpfr::igamma(a[i], 0)
#>                         c3 = ((exp(-w[i]) * w[i]^Rmpfr::mpfr(a[i],
#>                           128))/Rmpfr::igamma(a[i], 0))
#>                         I_term[i] = Rmpfr::asNumeric(c1/c2 +
#>                           c3 * (c4 + (1/(2 * b[i])^2) * (c5 -
#>                             c6 + c7 - w[i]^3/2)))
#>                       }
#>                       else {
#>                         c2 = gamma(a[i])
#>                         c3 = ((exp(-w[i]) * w[i]^a[i])/gamma(a[i]))
#>                         I_term[i] = (c1/c2 + c3 * (c4 + (1/(2 *
#>                           b[i])^2) * (c5 - c6 + c7 - w[i]^3/2)))
#>                       }
#>                     }
#>                     coeff1 = (2 * n - 1) * I_term[5]/(6 * sqrt(2 *
#>                       n * pi)) - (n - 1) * I_term[6]/(3 * sqrt(2 *
#>                       n * pi))
#>                     coeff2 = (n - 1) * I_term[1]/24 - (n - 1) *
#>                       (n + 2) * I_term[2]/(12 * n) + (n + 4) *
#>                       (n - 1) * I_term[3]/(24 * n)
#>                     coeff3 = (n - 1) * (2 * n + 5) * I_term[1]/72 -
#>                       (n - 1) * (2 * n^2 + 5 * n + 8) * I_term[2]/(24 *
#>                         n) + (n - 1) * (2 * n^2 + 5 * n + 12) *
#>                       I_term[3]/(24 * n) - (n - 1) * (2 * n^2 +
#>                       5 * n + 12) * I_term[4]/(72 * n)
#>                     new_prob[t] = 1 - (I_term[1]/2 + S * coeff1 -
#>                       K * coeff2 + S^2 * coeff3)
#>                   }
#>                   good_tscore = which(abs(new_prob - significance) <
#>                     1e-04)
#>                   new_tscore = tstat_list[good_tscore[which.min(abs(good_tscore -
#>                     which.min(abs(tstat_list - closest_t))))]]
#>                 }
#>             }
#>             Skewed_UPL = emission_mean + new_tscore * sqrt(var.s *
#>                 (1/n + 1/future_runs))
#>         }
#>     }
#>     return(Skewed_UPL)
#> }
#> <bytecode: 0x0000016f3165b5f8>
#> <environment: namespace:UPLforOAR>
```

obs_density()

```
#> function (data, emissions, xvals = NULL, up = Inf, low = 0, kernel = "gamma",
#>     bw = NULL)
#> {
#>     data_temp = tibble::tibble(emissions = data[[emissions]])
#>     sigma = stats::sd(data_temp$emissions)
```

```
#>     if (sigma == 0) {
#>         stop("Cannot find density for zero variance data")
#>     }
#>     if (is.null(bw)) {
#>         bw = sigma * nrow(data_temp)^(-2/5)
#>     }
#>     if (is.null(xvals)) {
#>         xvals = seq(0, 3 * max(data_temp$emissions), length.out = 1024)
#>     }
#>     if (is.numeric(bw)) {
#>         Obs_onPoint = np::npuniden.boundary(X = data_temp$emissions,
#>             Y = data_temp$emissions, a = low, b = up, proper = TRUE,
#>             kertype = kernel, h = bw)
#>         obs_den_df = np::npuniden.boundary(X = data_temp$emissions,
#>             Y = xvals, a = low, b = up, proper = TRUE, kertype = kernel,
#>             h = bw)
#>     }
#>     else if (is.character(bw)) {
#>         Obs_onPoint = np::npuniden.boundary(X = data_temp$emissions,
#>             Y = data_temp$emissions, bwmethod = bw, a = low,
#>             b = up, proper = TRUE, kertype = kernel)
#>         obs_den_df = np::npuniden.boundary(X = data_temp$emissions,
#>             Y = xvals, a = low, b = up, proper = TRUE, kertype = kernel,
#>             bwmethod = bw)
#>     }
#>     obs_den_df = tibble::tibble(x_hat = xvals, ydens = obs_den_df$f)
#>     Obs_onPoint = tibble::tibble(emissions = data_temp$emissions,
#>         ydens = Obs_onPoint$f)
#>     test_int = sfsmisc::integrate.xy(obs_den_df$x_hat, obs_den_df$ydens)
#>     if (abs(test_int - 1) > 0.05) {
#>         warning("density distribution does not integrate to 1,\n            consider adjusting xvals
#>     }
#>     output = list(Obs_onPoint = Obs_onPoint, obs_den_df = obs_den_df)
#>     return(output)
#> }
#> <bytecode: 0x0000016f331b7db8>
#> <environment: namespace:UPLforOAR>
```

Appendix D: System and Package Versions

```
#> R version 4.4.0 (2024-04-24 ucrt)
#> Platform: x86_64-w64-mingw32/x64
#> Running under: Windows 11 x64 (build 22631)
#>
#> Matrix products: default
#>
#>
#> locale:
#> [1] LC_COLLATE=English_United States.utf8
#> [2] LC_CTYPE=English_United States.utf8
#> [3] LC_MONETARY=English_United States.utf8
#> [4] LC_NUMERIC=C
#> [5] LC_TIME=English_United States.utf8
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] grid      stats     graphics  grDevices utils     datasets  methods
#> [8] base
#>
#> other attached packages:
#>  [1] UPLforOAR_0.1.0   sfsmisc_1.1-22    np_0.60-18        EnvStats_3.1.0
#>  [5] lubridate_1.9.4   forcats_1.0.1     stringr_1.5.2     purrr_1.1.0
#>  [9] readr_2.1.5       tidyr_1.3.1       tibble_3.3.0      tidyverse_2.0.0
#> [13] ggplot2_4.0.0     dplyr_1.1.4       htmltools_0.5.8.1
#>
#> loaded via a namespace (and not attached):
#>  [1] generics_0.1.4     stringi_1.8.7      lattice_0.22-6     hms_1.1.3
#>  [5] cubature_2.1.4     digest_0.6.37      magrittr_2.0.4     evaluate_1.0.5
#>  [9] timechange_0.3.0   RColorBrewer_1.1-3 fastmap_1.2.0      Matrix_1.7-0
#> [13] survival_3.5-8     scales_1.4.0       cli_3.6.5          crayon_1.5.3
#> [17] rlang_1.1.6        bit64_4.6.0-1      splines_4.4.0      withr_3.0.2
#> [21] yaml_2.3.10        parallel_4.4.0     tools_4.4.0        SparseM_1.84-2
#> [25] tzdb_0.5.0         MatrixModels_0.5-4 boot_1.3-30        vctrs_0.6.5
#> [29] R6_2.6.1           lifecycle_1.0.4    bit_4.6.0          vroom_1.6.6
#> [33] MASS_7.3-60.2      pkgconfig_2.0.3    pillar_1.11.1      gtable_0.3.6
#> [37] glue_1.8.0         Rcpp_1.1.0         xfun_0.53          tidyselect_1.2.1
#> [41] rstudioapi_0.17.1  knitr_1.50         farver_2.1.2       patchwork_1.3.2
#> [45] labeling_0.4.3     rmarkdown_2.30     compiler_4.4.0     quadprog_1.5-8
#> [49] quantreg_6.1       S7_0.2.0
```

Appendix E: Rmd Code Chunks

```
# Below is an example of loading data, sorting and selecting to top performers,
# and calculating the correct distribution and UPL for existing source and NSPS.
# If there are multiple HAP or subcategories, simply repeat with code block for
# those data, making sure to use unique names so you don't overwrite the results.
if (params$CAA_section == 112){
  CAA_text = 'CAA Section 112(d)(3)'
} else if (params$CAA_section == 129){
  CAA_text = 'CAA Section 129(a)(2)'
}
dat_emiss = read_csv(params$data)
if (any(dat_emiss$emissions == 0)){
  zero_warn = TRUE
} else {
  zero_warn = FALSE
}
dat_top_exist = MACT_existing(CAA_section = params$CAA_section, data = dat_emiss,
                              emissions = 'emissions', sources = 'sources')
dat_top_new = MACT_new(data = dat_emiss, emissions = 'emissions',
                       sources = 'sources')
dat_top_exist$ln_emiss = log(dat_top_exist$emissions)
dat_top_exist$ln_emiss = replace(dat_top_exist$ln_emiss,
                                 !is.finite(dat_top_exist$ln_emiss), NA)
dat_top_new$ln_emiss = log(dat_top_new$emissions)
dat_top_new$ln_emiss = replace(dat_top_new$ln_emiss,
                               !is.finite(dat_top_new$ln_emiss), NA)
dat_topmeans_exist = dat_top_exist %>% group_by(sources) %>%
                              summarize(means = mean(emissions), counts = n())
if (all(dat_topmeans_exist$counts == 1)){
  source_fig = FALSE
} else {
  source_fig = TRUE
}
best_source = as.character(dat_top_new$sources)[1]
distribution_result_exist = distribution_type(data = dat_top_exist,
                                              emissions = 'emissions')
distribution_result_new = distribution_type(data = dat_top_new,
                                            emissions = 'emissions')

if (distribution_result_exist == "Normal"){
  UPL_exist = Normal_UPL(data = dat_top_exist,
                         emissions = 'emissions',
                         significance = params$significance,
                         future_runs = params$future_runs)
} else if (distribution_result_exist == "Lognormal"){
  UPL_exist = Lognormal_UPL(data = dat_top_exist,
                            emissions = 'emissions',
                         significance = params$significance,
                         future_runs = params$future_runs)
} else if (distribution_result_exist == "Skewed"){
  UPL_exist = Skewed_UPL(data = dat_top_exist,
                         emissions = 'emissions',
                         significance = params$significance,
```

```
                        future_runs = params$future_runs)
}

if (distribution_result_new == "Normal"){
  UPL_new = Normal_UPL(data = dat_top_new,
                       emissions = 'emissions',
                       significance = params$significance,
                       future_runs = params$future_runs)
} else if (distribution_result_new == "Lognormal"){
  UPL_new = Lognormal_UPL(data = dat_top_new,
                          emissions = 'emissions',
                          significance = params$significance,
                          future_runs = params$future_runs)
} else if (distribution_result_new == "Skewed"){
  UPL_new = Skewed_UPL(data = dat_top_new,
                       emissions = 'emissions',
                       significance = params$significance,
                       future_runs = params$future_runs)
}
col1=c('Subcategory 1','Subcategory 2','Subcategory 3','Subcategory 4',
       'Subcategory 5')
col2=c('HAP 1, HAP 2', 'HAP 1', 'HAP 1, HAP 2, HAP 3', 'None', 'None')
col3=c('HAP 1, HAP 2', 'HAP 1', 'HAP 1, HAP 2, HAP 3', 'None', 'None')
col4=c('HAP 1, HAP 2, X, Y, Z', 'HAP 1, X, Y, Z', 'HAP 1, HAP 2, HAP 3, X, Y, Z',
       'None', 'X, Y, Z')
table1=tibble(col1=col1,col2=col2,col3=col3,col4=col4)
table1%>%knitr::kable(
  caption="Here is an example of a table in case you need one",
             col.names=c('',
                         'Existing MACT floor (old units)',
                         'Existing MACT floor (new units)',
                         'New MACT floor'))
ggplot(data = dat_top_exist, aes(emissions, fill = sources))+
          geom_density(alpha = 0.5, bounds = c(0, Inf))+
          ylab("Density")+xlab("Chemical X emissions (units)")+
          ggtitle("Subcategory A top sources test run densities")+
          multi_source_theme()+labs(fill = 'Top sources',
                                    color = 'Top sources')+
          geom_vline(aes(xintercept = means, color = sources),
             data = dat_topmeans_exist, linewidth = 1)+
          guides(fill = guide_legend(nrow = 2, byrow = TRUE),
                color = guide_legend(nrow = 2, byrow = TRUE))+
          scale_x_continuous(expand = expansion(mult = c(0, 0.05)))+
          scale_y_continuous(expand = expansion(mult = c(0, 0.05)))+
          coord_cartesian(clip = 'off')+
          geom_rug(sides = 'b', aes(x = emissions, color = sources),
                   data = dat_top_exist, alpha = 0.5, outside = TRUE)
x_hat = seq(0, 3*max(dat_top_exist$emissions), length.out = 1024)
obs_dens_results = obs_density(dat_top_exist, xvals = x_hat,
                               emissions = 'emissions')
Obs_onPoint = obs_dens_results$Obs_onPoint
obs_den_df = obs_dens_results$obs_den_df
if (distribution_result_exist == "Normal"){
```

```
  pdf_n = dnorm(x_hat, mean = mean(dat_top_exist$emissions, na.rm = T),
              sd = sd(dat_top_exist$emissions, na.rm = T))
  pred_dat = tibble(x_hat, pdf_n)
} else if (distribution_result_exist == "Lognormal"){
  pdf_n = dlnorm(x_hat,mean = log(mean(dat_top_exist$emissions, na.rm = T)),
              sd = sd(log(dat_top_exist$emissions), na.rm = T))
  pred_dat = tibble(x_hat, pdf_n)
} else {
  pred_dat = tibble(xhat = rep(NA, 1024), pdf_n = rep(NA, 1024))
}
ggplot()+
  geom_line(data = obs_den_df, size = 0.75,
            aes(y = ydens, x = x_hat, color = 'a'))+
  geom_area(data = obs_den_df, alpha = 0.25,
            aes(y = ydens, x = x_hat, fill = 'a'))+
  geom_point(aes(y = ydens, x = emissions), data = Obs_onPoint,
             size = 3, alpha = 0.5, shape = 19, color = 'black')+
  geom_line(aes(y = pdf_n, x = x_hat, color = 'b'),
               data = pred_dat, size = 0.75, linetype = 2)+
  geom_area(aes(y = pdf_n, x = x_hat, fill = 'b'), alpha = 0.25,
            data = pred_dat)+
  ylab("Density")+xlab("Chemical X emissions (units)")+
  ggtitle("Subcategory A existing source\nMACT floor test run population")+
  pop_distr_theme()+
  geom_vline(aes(xintercept = mean(dat_top_exist$emissions),
                 color = 'a'), linewidth = 1, linetype = 1)+
  geom_vline(aes(xintercept = UPL_exist, color = 'b'),
             linewidth = 1, linetype = 2)+
  scale_x_continuous(expand = expansion(mult = c(0, 0.05)))+
  scale_y_continuous(expand = expansion(mult = c(0, 0.05)))+
  coord_cartesian(clip = 'off')+
  labs(color = 'Distribution:', fill = 'Distribution:')+
  geom_rug(sides = 'b', aes(x = emissions), data = dat_top_exist,
           alpha = 0.5, outside = TRUE, color = 'black')+
  scale_color_manual(values = c('black', '#FF7F00'),
                     labels = c('Observations', distribution_result_exist))+
  scale_fill_manual(values = c('black', '#FF7F00'),
                    labels = c('Observations', distribution_result_exist))
x_hat = seq(0, 3*max(dat_top_new$emissions), length.out = 1024)
obs_dens_results = obs_density(dat_top_new, xvals = x_hat,
                               emissions = 'emissions')
Obs_onPoint = obs_dens_results$Obs_onPoint
obs_den_df = obs_dens_results$obs_den_df
if (distribution_result_new == "Normal"){
  pdf_n = dnorm(x_hat, mean = mean(dat_top_new$emissions, na.rm = T),
              sd = sd(dat_top_new$emissions, na.rm = T))
  pred_dat = tibble(x_hat, pdf_n)
} else if (distribution_result_new == "Lognormal"){
  pdf_n = dlnorm(x_hat,mean = log(mean(dat_top_new$emissions, na.rm = T)),
              sd = sd(log(dat_top_new$emissions), na.rm = T))
  pred_dat = tibble(x_hat, pdf_n)
} else {
  pred_dat = tibble(xhat = rep(NA, 1024), pdf_n = rep(NA, 1024))
```

```
}
ggplot()+
  geom_line(data = obs_den_df, size = 0.75,
            aes(y = ydens, x = x_hat, color = 'a'))+
  geom_area(data = obs_den_df, alpha = 0.25,
            aes(y = ydens, x = x_hat, fill = 'a'))+
  geom_point(aes(y = ydens, x = emissions), data = Obs_onPoint,
             size = 3, alpha = 0.5, shape = 19, color = 'black')+
  geom_line(aes(y = pdf_n, x = x_hat, color = 'b'),
               data = pred_dat, size = 0.75, linetype = 2)+
  geom_area(aes(y = pdf_n, x = x_hat, fill = 'b'), alpha = 0.25,
            data = pred_dat)+
  ylab("Density")+xlab("Chemical X emissions (units)")+
  ggtitle("Subcategory A new source\nMACT floor test run population")+
  pop_distr_theme()+
  geom_vline(aes(xintercept = mean(dat_top_new$emissions),
                 color = 'a'), linewidth = 1, linetype = 1)+
  geom_vline(aes(xintercept = UPL_new, color = 'b'),
             linewidth = 1, linetype = 2)+
  scale_x_continuous(expand = expansion(mult = c(0, 0.05)))+
  scale_y_continuous(expand = expansion(mult = c(0, 0.05)))+
  coord_cartesian(clip = 'off')+
  labs(color = 'Distribution:', fill = 'Distribution:')+
  geom_rug(sides = 'b', aes(x = emissions), data = dat_top_new,
           alpha = 0.5, outside = TRUE, color = 'black')+
  scale_color_manual(values = c('black', '#377EB8'),
                     labels = c('Observations', distribution_result_new))+
  scale_fill_manual(values = c('black', '#377EB8'),
                    labels = c('Observations', distribution_result_new))
dat_UPL = tibble(HAP = c("Chemical X"),
                 Subcategory = c("Type A"),
                 Existing = c(signif(UPL_exist,4)),
                 New = c(signif(UPL_new, 4)))
dat_UPL %>% knitr::kable(
  caption = "Upper Predictive Limit (UPL) results for existing and
  new source MACT floors in (measurement units)", digits = 12,
                         col.names = c("HAP", "Subcategory",
                                       "Existing source UPL",
                                       "New source UPL"))
dat_top_exist %>% knitr::kable(
  caption = "Top performering source emissions data used in existing source
  UPL calculations.", digits = 12, col.names = names(dat_top_exist))
dat_top_new %>% knitr::kable(
  caption = "Best performering source emissions data used in new source
  UPL calculations.", digits = 12, col.names = names(dat_top_new))
```